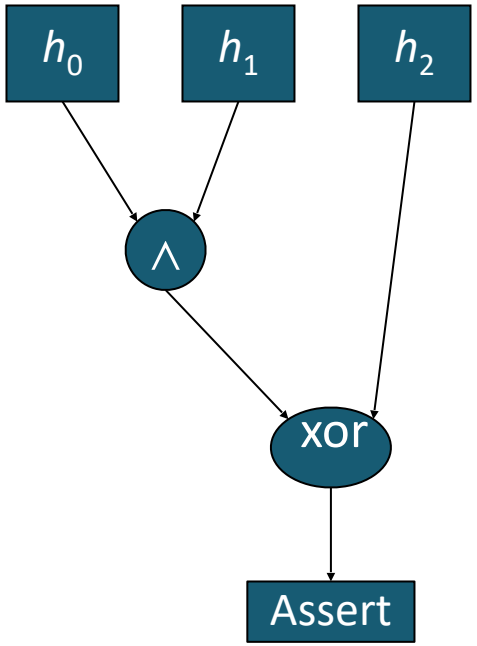
Lecture 9
Solving Constraints

Armando Solar-Lezama

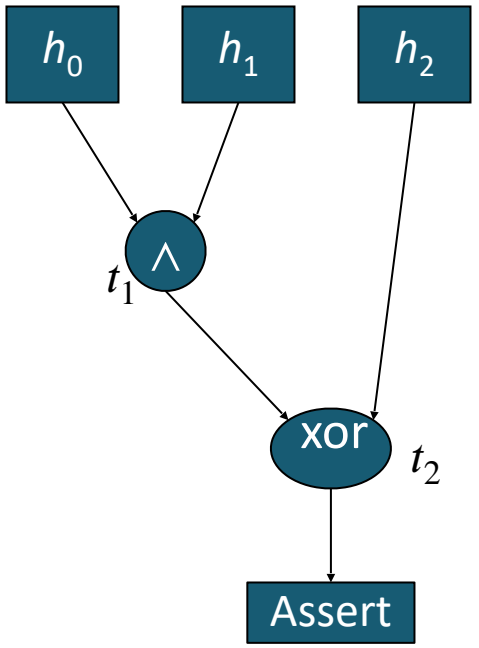
Constraints to SAT



SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

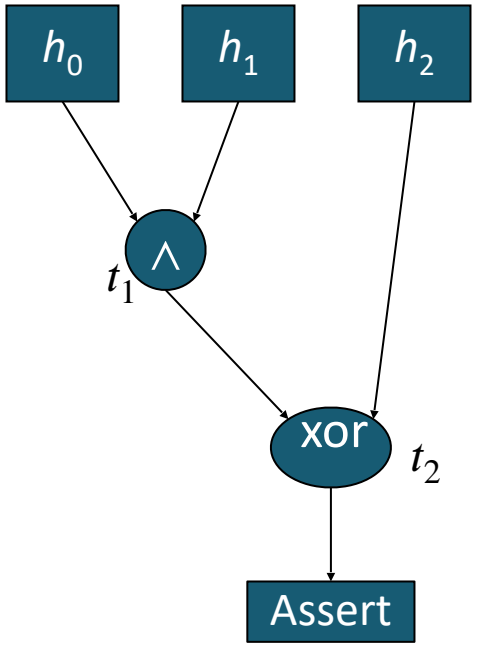
Constraints to SAT



SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

Constraints to SAT

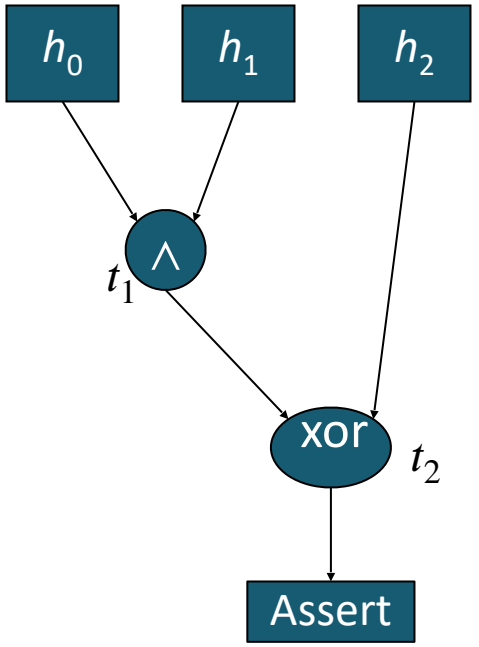


$$h_0 \wedge h_1 \Rightarrow t_1 t_1 \Rightarrow h_0 t_1 \Rightarrow h_1$$

SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

Constraints to SAT



$$h_0 \wedge h_1 \Rightarrow t_1 t_1 \Rightarrow h_0 t_1 \Rightarrow h_1$$

SAT Solver only understands CNF

- Sum (OR) of variables and their negation
- Equivalent to $\bigwedge_{i \in X} l_i \Rightarrow l_j$

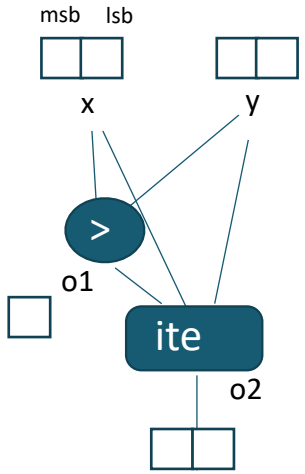
$$t_1 \wedge h_2 \Rightarrow \bar{t_2}\bar{t_1} \wedge \bar{h_2} \Rightarrow \bar{t_2} t_1 \wedge \bar{h_2} \Rightarrow t_2 \bar{t_1} \wedge h_2 \Rightarrow t_2$$

What about integers

Standard approach is bit-blasting

Eager bit-blasting to CNF

SMT solver High-level constraint



SAT solver CNF clauses

$$\overline{y_1} \vee x_1 \vee \overline{t_1} \qquad \overline{t_1} \vee y_0 \vee \overline{x_0} \vee t_2$$
$$y_1 \vee \overline{x_1} \vee \overline{t_1} \qquad \overline{t_3} \vee \overline{y_1}$$
$$\overline{y_1} \vee \overline{x_1} \vee t_1 \qquad \overline{t_3} \vee x_1$$
$$y_1 \vee x_1 \vee t_1 \qquad y_1 \vee \overline{x_1} \vee t_3$$
$$\overline{t_2} \vee t_1 \qquad o_1 \vee \overline{t_2}$$
$$\overline{t_2} \vee \overline{y_0} \qquad o_1 \vee \overline{t_3}$$
$$\overline{t_2} \vee x_0 \qquad t_2 \vee t_3 \vee \overline{o_1}$$

$$\overline{o_1} \vee x_1 \vee \overline{o_{2_1}} \qquad \overline{o_1} \vee x_0 \vee \overline{o_{2_0}}$$
$$\overline{o_1} \vee \overline{x_1} \vee o_{2_1} \qquad \overline{o_1} \vee \overline{x_0} \vee o_{2_0}$$
$$o_1 \vee y_1 \vee \overline{o_{2_1}} \qquad o_1 \vee y_0 \vee \overline{o_{2_0}}$$
$$o_1 \vee \overline{y_1} \vee o_{2_1} \qquad o_1 \vee \overline{y_0} \vee o_{2_0}$$
$$x_1 \vee y_1 \vee \overline{o_{2_1}} \qquad x_0 \vee y_0 \vee \overline{o_{2_0}}$$
$$\overline{x_1} \vee \overline{y_1} \vee o_{2_1} \qquad \overline{x_0} \vee \overline{y_0} \vee o_{2_0}$$
$$\ldots$$

Simple approach: Individually Generate CNF Clauses for each operator

Embelishments

Go through an intermediate representation

- And Inverter Graphs (AIG) are a popular IR
  - Allow additional algebraic simplification/structural hashing
  - Bit-blasting is not confined to individual operator boundaries

Embelishments

Go through an intermediate representation

- And Inverter Graphs (AIG) are a popular IR
  - Allow additional algebraic simplification/structural hashing
  - Bit-blasting is not confined to individual operator boundaries

Bit-blast lazily

- Start with an predicate $P'(x)$ such that $P'(x) \Rightarrow P(x)$
- The idea is for P' to be simpler, make it easy to find a solution.
  - Examples include reduced bit-width
- Can also use an underapproximation $P(x) \Rightarrow P_u(x)$ to whelp with the unsat case

One Hot encoding

Basically Unary encoding

- One bit per possible value

$$[(5, b_1), (7, b_2), (9, b_3)] \quad + \quad [(3, b_4), (5, b_5), (7, b_6)]$$



$$[(8, b_1 b_4), (10, b_1 b_5 \vee b_2 b_4),$$
$$(12, b_1 b_6 \vee b_5 b_2 \vee b_3 b_4), (14, b_2 b_6 \vee b_3 b_5), (16, b_3 b_6)]$$

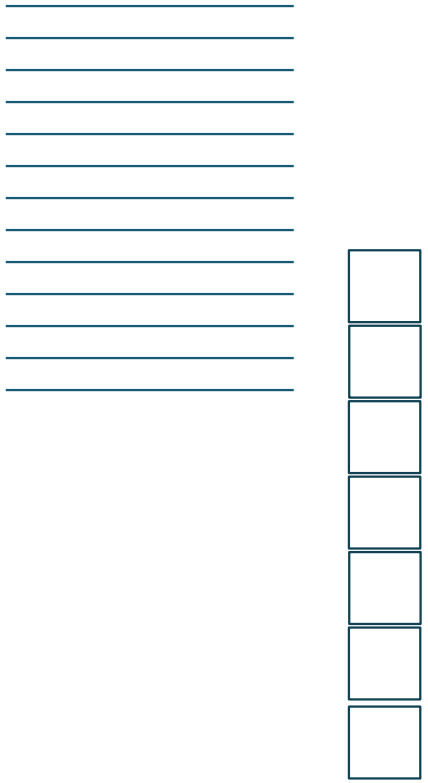
One Hot encoding

Can be very efficient, but can also blow up

SAT Solver

SAT solving in a nutshell

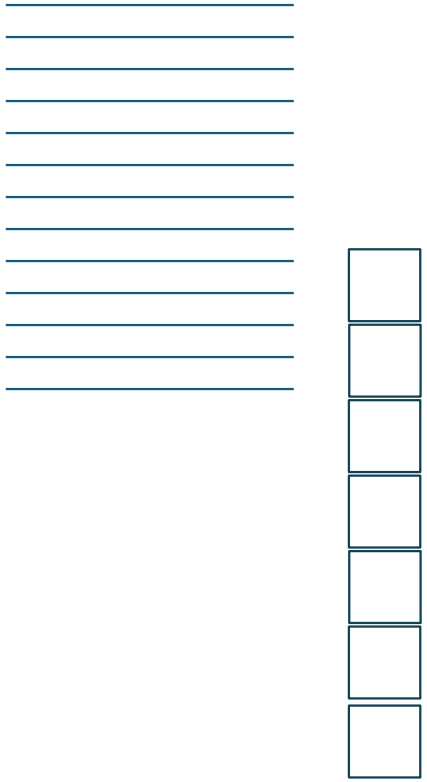
Constraint database



SAT Solver

SAT solving in a nutshell

Constraint database



Constraint is a CNF Clause

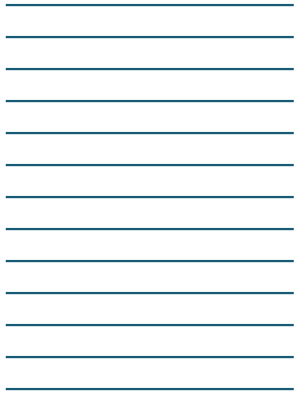
$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$



SAT Solver

SAT solving in a nutshell

Constraint database





x_1

Constraint is a CNF Clause

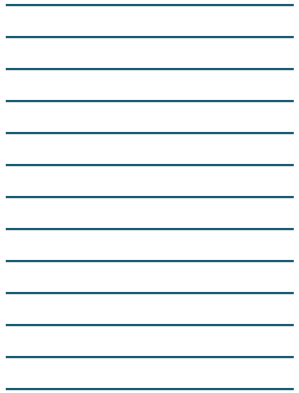
$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$



SAT Solver

SAT solving in a nutshell

Constraint database





x_1

$-x_1 \vee x_7$



x_7

Constraint is a CNF Clause

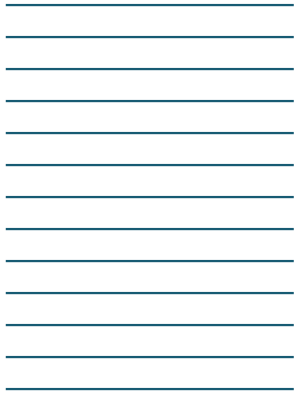
$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$



SAT Solver

SAT solving in a nutshell

Constraint database





x_1

$-x_1 \vee x_7$



x_7

x_5

Constraint is a CNF Clause

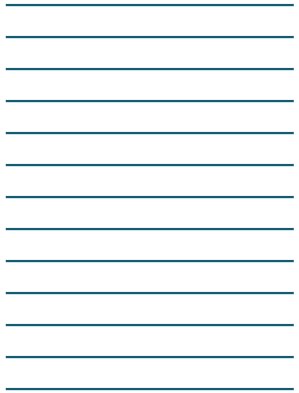
$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$



SAT Solver

SAT solving in a nutshell

Constraint database





x_1

$-x_1 \vee x_7$



x_7

$-x_7 \vee x_6 \vee -x_5$



x_6

x_5

Constraint is a CNF Clause

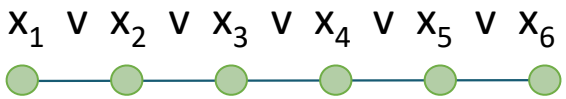
$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$



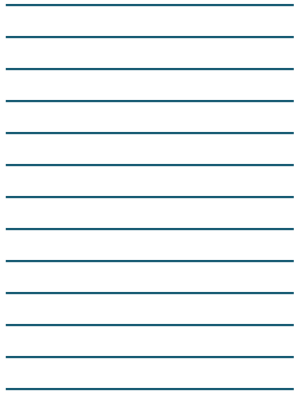
SAT Solver

SAT solving in a nutshell

Constraint is a CNF Clause



Constraint database





x_1

$-x_1$ v x_7



x_7

x_5

$-x_7$ v x_6 v $-x_5$



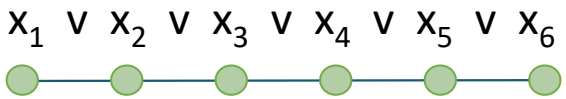
x_6

$-x_9$

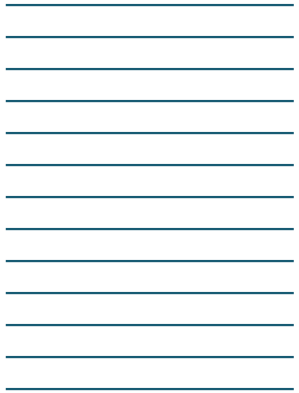
SAT Solver

SAT solving in a nutshell

Constraint is a CNF Clause

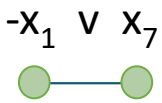


Constraint database

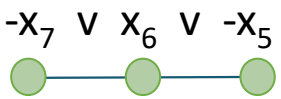




x_1

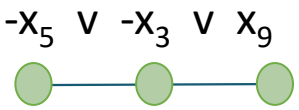


x_7



x_6

x_5



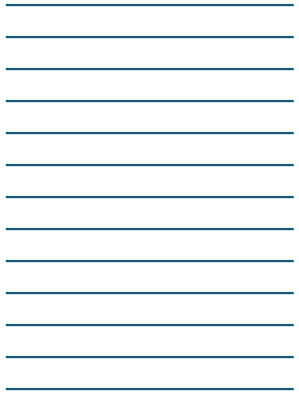
$-x_3$

$-x_9$

SAT Solver

SAT solving in a nutshell

Constraint database





x_1

$-x_1 \vee x_7$

x_7

x_5

$-x_7 \vee x_6 \vee -x_5$

$-x_5 \vee -x_3 \vee x_9$

$-x_9$

x_6 $-x_3$

$-x_6 \vee x_3 \vee x_4$

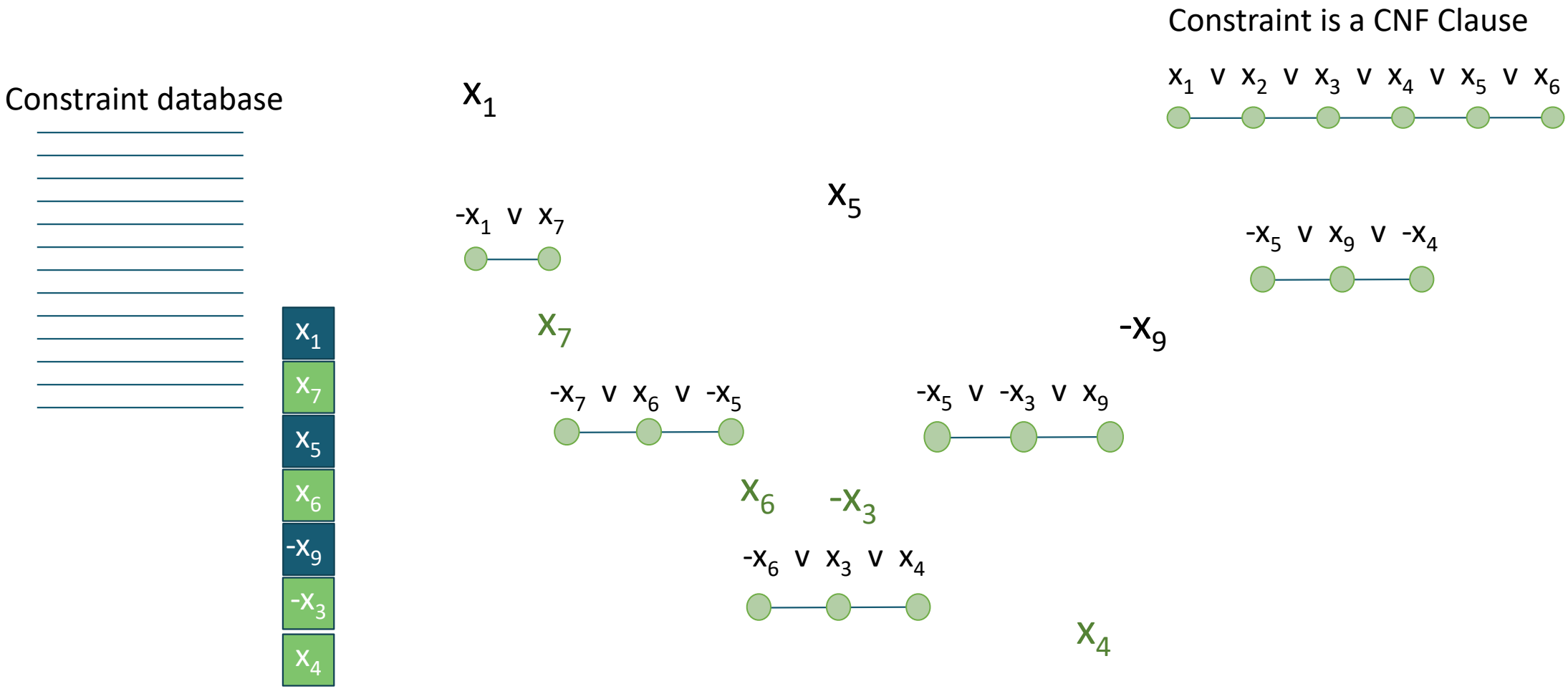
x_4

Constraint is a CNF Clause

$x_1 \vee x_2 \vee x_3 \vee x_4 \vee x_5 \vee x_6$

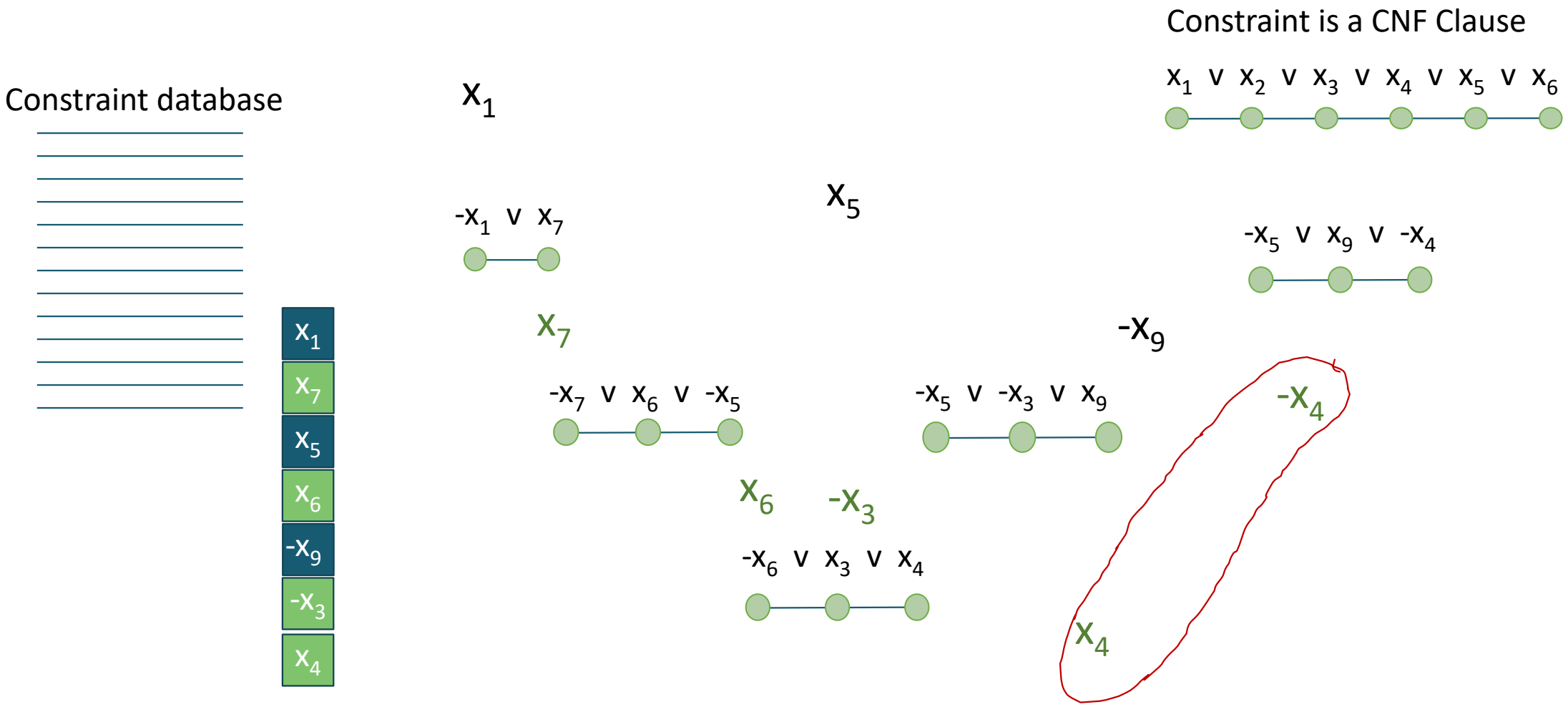
SAT Solver

SAT solving in a nutshell



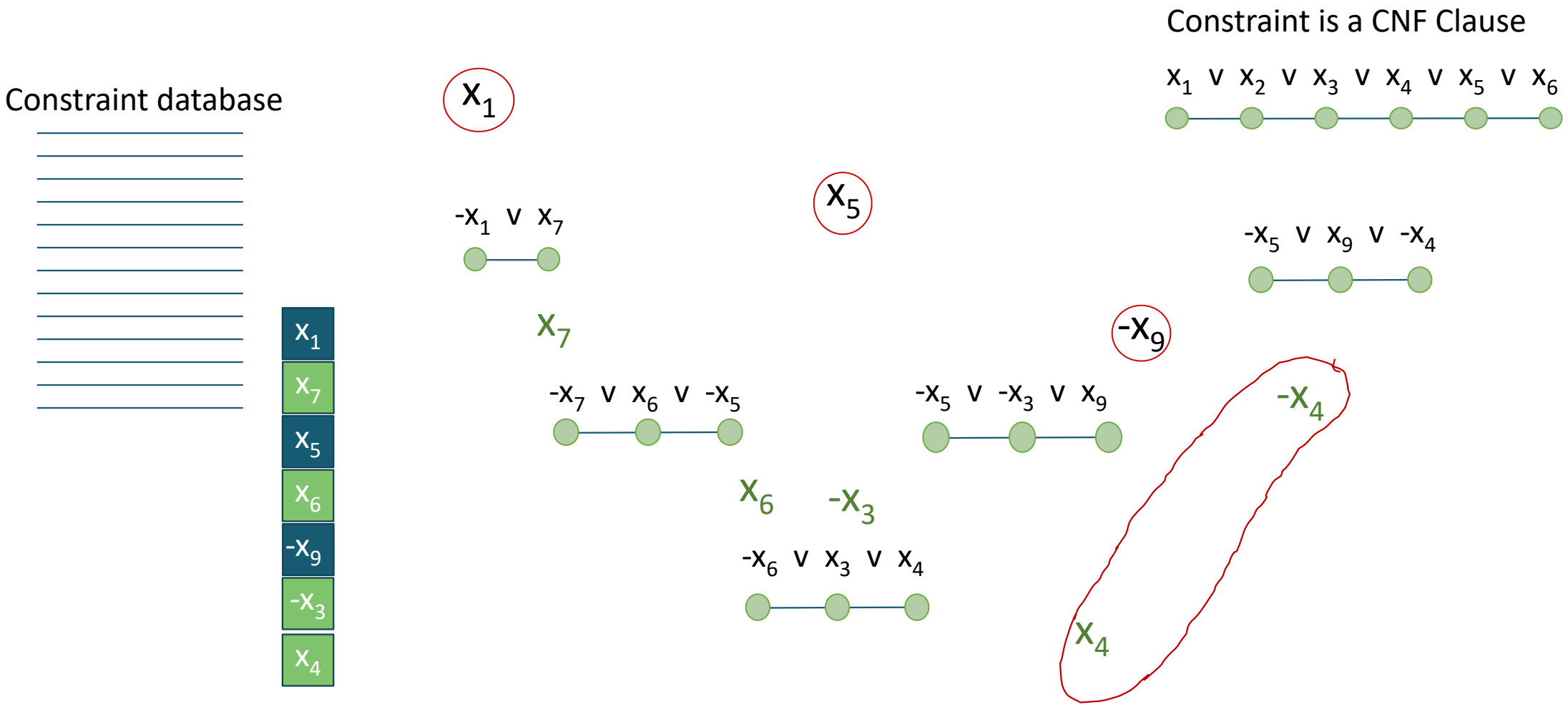
SAT Solver

SAT solving in a nutshell



SAT Solver

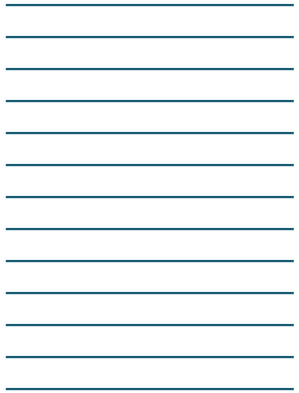
SAT solving in a nutshell



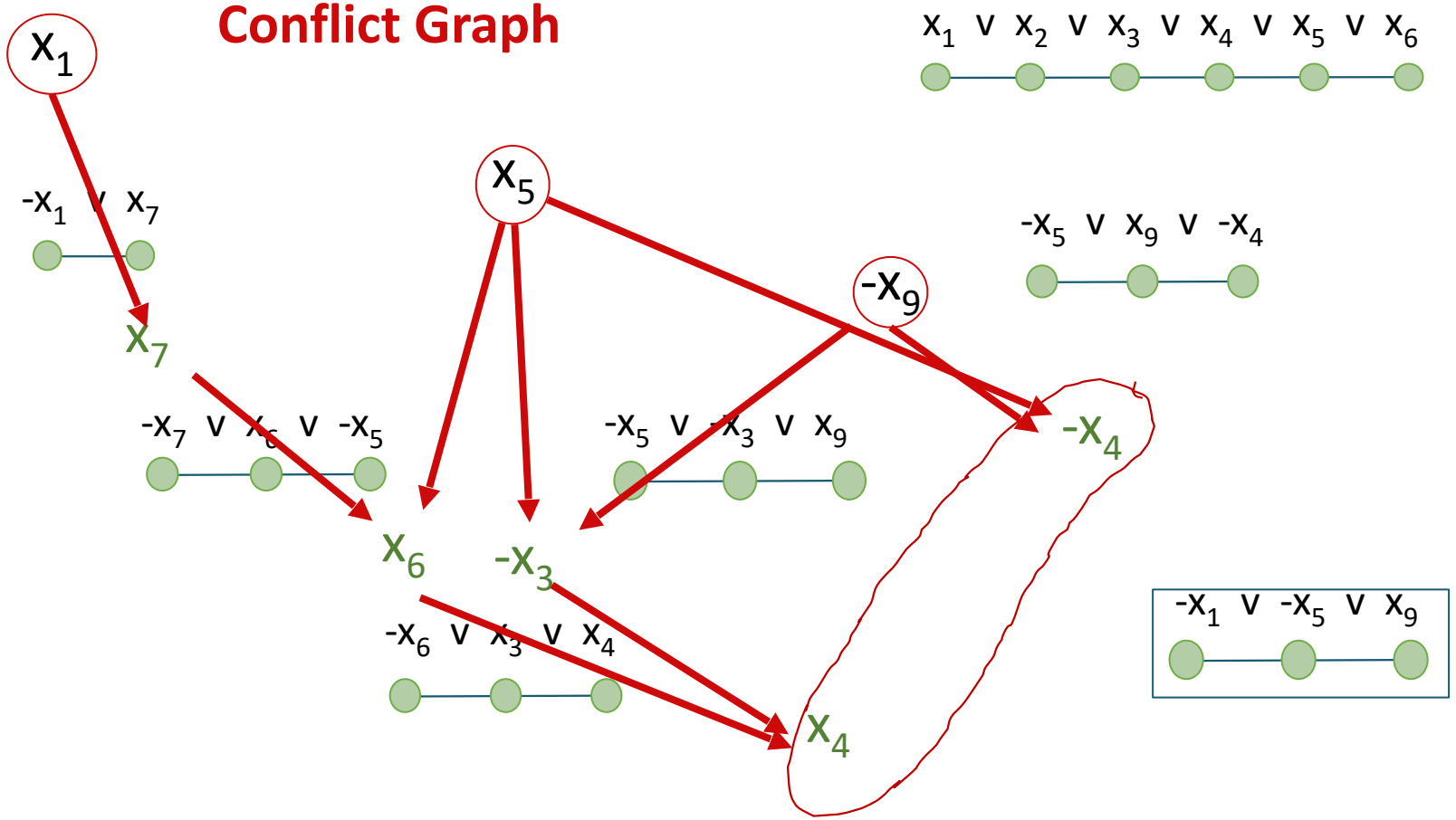
SAT Solver

SAT solving in a nutshell

Constraint database



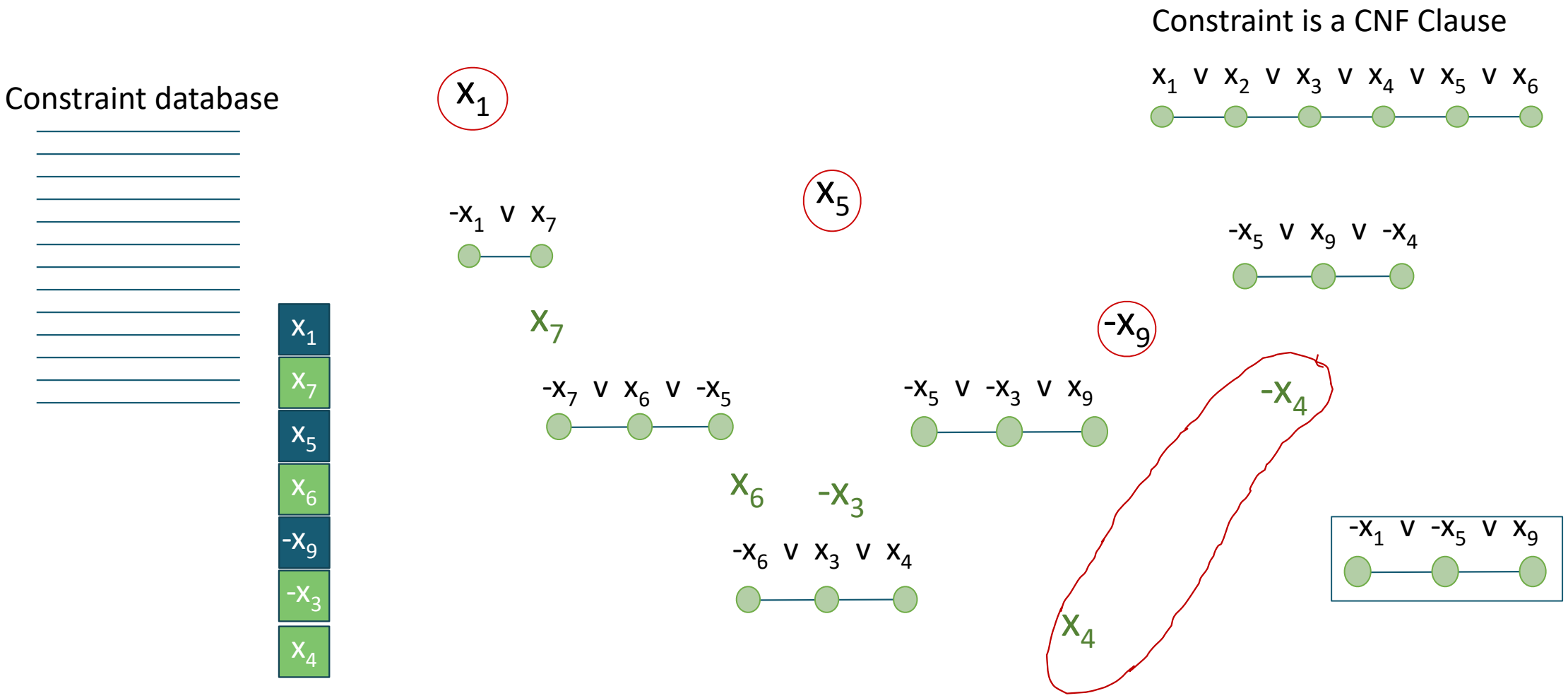




Constraint is a CNF Clause

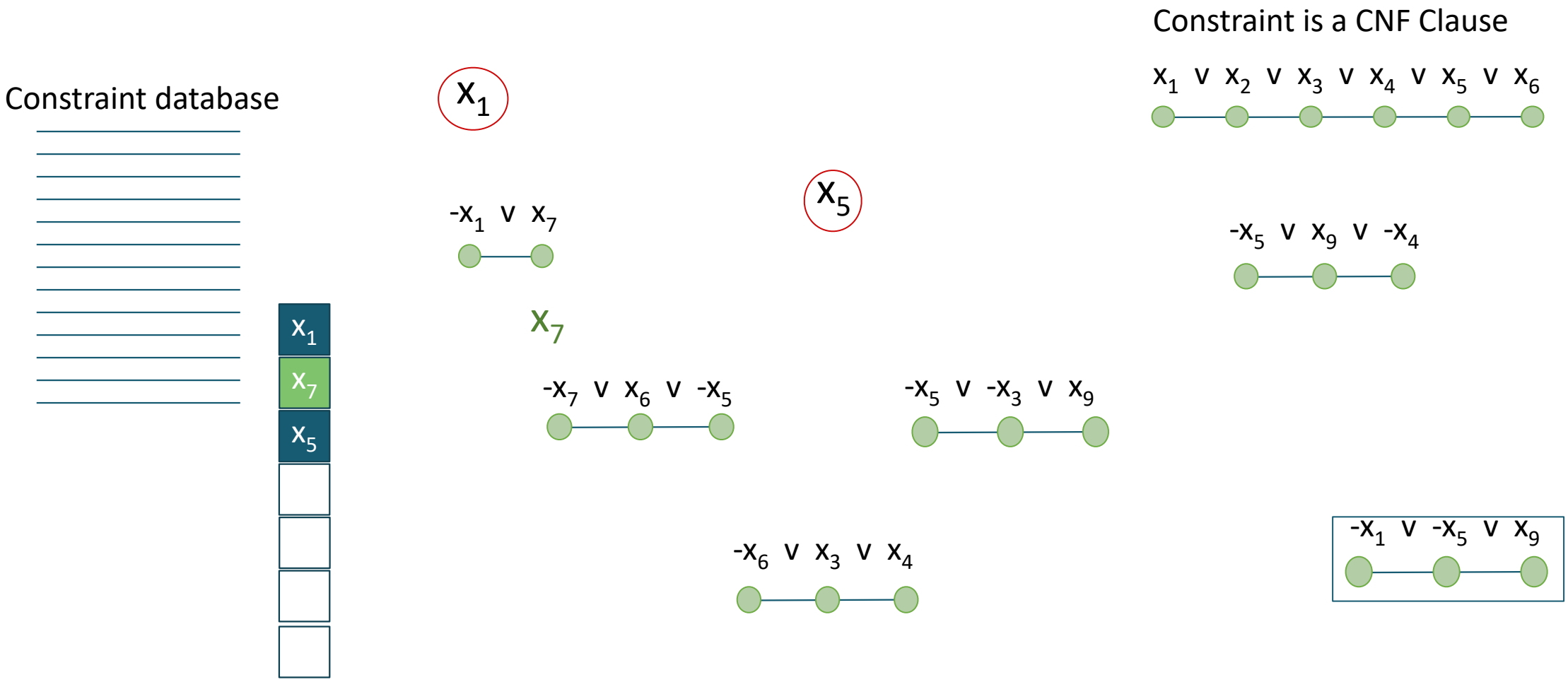
SAT Solver

SAT solving in a nutshell



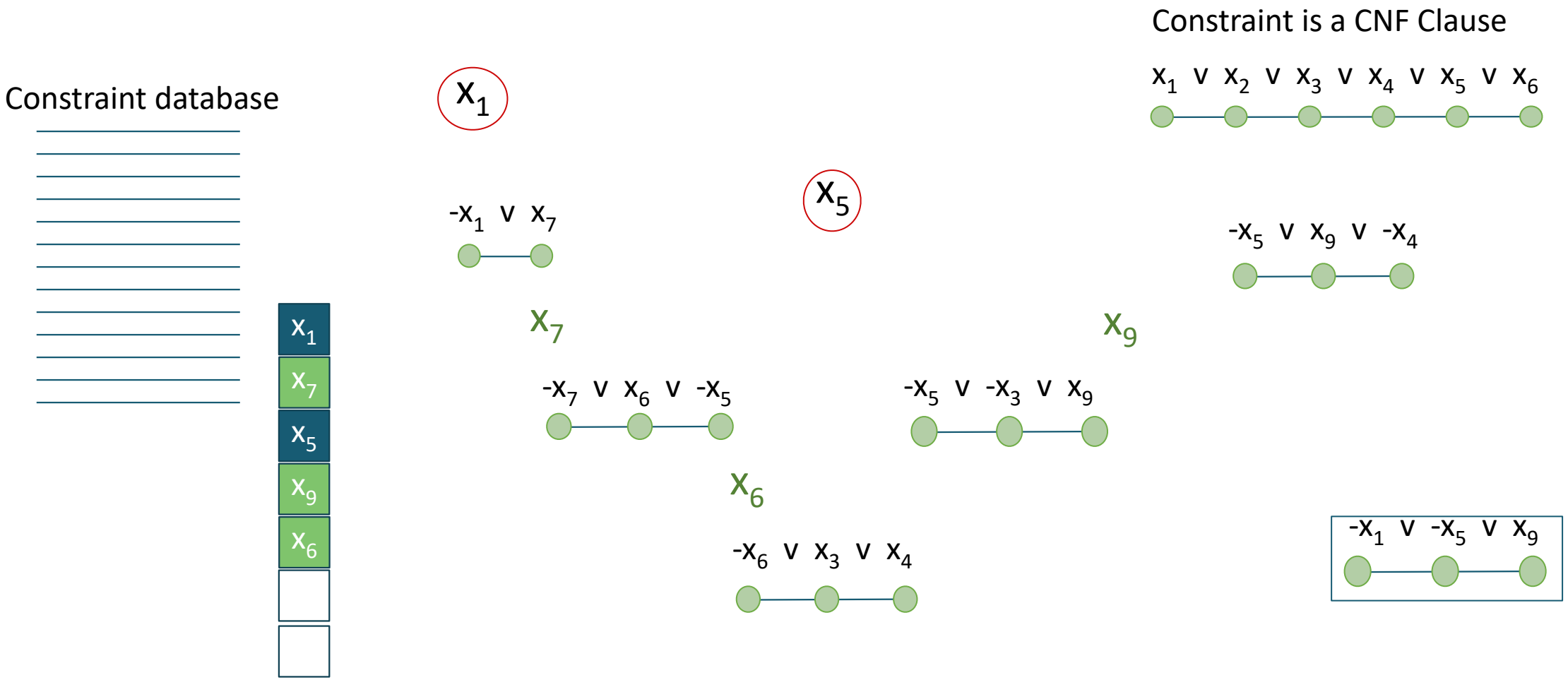
SAT Solver

SAT solving in a nutshell



SAT Solver

SAT solving in a nutshell

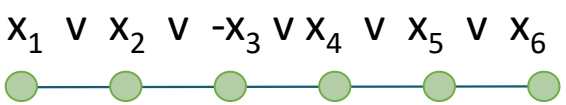


Key questions

How do we find all the relevant clauses after a variable is set

How do we know which variable to try next

Two-literal watching



Propagation only happens when the second to last literal is set

- visiting the clause when this is not the case is a waste of time

Watch two unset literals.

- when others are set, ignore

VSIDS

Variable State Independent Decaying Sum

- Additive bumping and multiplicative decay
- Bump when a variable participates in a conflict
- Imprecise but very very fast

What about Arithmetic?

1) Bit-blast

2) Unary encoding

3) SMT